

Web Security

Basic to Expert



Core Methods

- Password hashing (bcrypt, argon2)
- Sessions & secure cookies (HttpOnly, SameSite, Secure)
- JWT, OAuth 2.0, OpenID Connect
- MFA / OTP authentication
- Input validation (Joi, Zod)
- Prevent XSS, CSRF, SQLi/NoSQLi
- File upload/download security
- CORS and API security
- Rate limiting & brute force protection
- Helmet, CSP, HTTPS, HSTS
- Secrets management (.env, never in repo)
- Secure logging (mask sensitive info)
- Dependency scanning (npm audit, Snyk, Dependabot)
- Monitoring (Winston, Morgan, ELK)



hexsecteam



hexsec_tools



hexsecteam

Foundation

CIAt riad: Confidentiality, Integrity, Availability.

Least privilege: Minimum Access that you need.

Defense in depth: Multiple layers of security (firewalls, input validation, hashing, monitoring).

Zero trust: Don't assume any user or request is safe by default.



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



Introduction

It is about protecting websites and web apps from attacks that try to steal data, damage systems, or trick users.

It protects user data and builds trust.

It prevents downtime.

It saves from legal issues.



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



JWT Authentication

```
// Used for stateless authentication.
// Structure: header.payload.signature.
// Always use HS256 or RS256 with a strong secret key.
// Short expiry time(e.g., 15 minutes) with refresh tokens.
// Don't store JWTs in localStorage(XSS risk).Use HttpOnly cookies.

const jwt = require('jsonwebtoken');
const secret = "jwtSecretKey";

// Sign token
function generateToken(user) {
  return jwt.sign({ id: user.id, role: user.role }, secret, { expiresIn: "15m" });
}

// Verify token
function verifyToken(req, res, next) {
  const token = req.cookies.token; // Store in HttpOnly cookie
  if (!token) return res.status(401).send("Access Denied");

  jwt.verify(token, secret, (err, decoded) => {
    if (err) return res.status(403).send("Invalid Token");
    req.user = decoded;
    next();
  });
}
```



hexsecteam



hexsec_tools



hexsecteam

Sessions & Cookies

```
//Store only session IDs in cookies, not user info.  
const session = require('express-session');  
const express = require('express');  
const app = express();  
  
app.use(session({  
  secret: "superSecretKey",  
  resave: false,  
  saveUninitialized: false,  
  cookie: {  
    httpOnly: true,    // JS can't read it  
    secure: true,      // Only HTTPS  
    sameSite: 'strict' // Prevent CSRF  
  }  
}));
```



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



Password Hashing

```
const bcrypt = require('bcrypt');

// Always hash passwords, never store plaintext
async function hashPassword(password) {
  const saltRounds = 12; // add these to avoid rainbow table attacks
  const hashed = await bcrypt.hash(password, saltRounds);
  return hashed;
}

//bonus: use pepper for extra strength
async function checkPassword(password, hash) {
  return await bcrypt.compare(password, hash);
}

// Usage
(async () => {
  const hash = await hashPassword("mySecret123");
  console.log("Hash:", hash);
  console.log("Match:", await checkPassword("mySecret123", hash)); // true
})();
```



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



OAuth (Google Login)

```
// Authorization Code Flow(secure, uses server).
//PKCE(Proof Key for Code Exchange) for mobile / SPAs.
// OpenID Connect adds identity info on top of OAuth.

const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;

passport.use(new GoogleStrategy({
  clientID: "GOOGLE_CLIENT_ID",
  clientSecret: "GOOGLE_CLIENT_SECRET",
  callbackURL: "/auth/google/callback"
}, (accessToken, refreshToken, profile, done) => {
  // Save user to DB if new
  done(null, profile);
}));

app.get('/auth/google',
  passport.authenticate('google', { scope: ['profile', 'email'] })
);

app.get('/auth/google/callback',
  passport.authenticate('google', { failureRedirect: '/' }),
  (req, res) => { res.redirect('/dashboard'); }
);
```



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



NoSQL Injection

Unsafe

```
js

// BAD: User input directly in query
db.query(`SELECT * FROM users WHERE email = '${req.body.email}'`);
```

Safe

```
js

// MySQL - use parameterized query
db.query("SELECT * FROM users WHERE email = ?", [req.body.email]);

// MongoDB with Mongoose
User.findOne({ email: req.body.email }); // safe if sanitized
```



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



CSRF Protection

```
//safe form submission
const csrf = require('csurf');
const cookieParser = require('cookie-parser');

app.use(cookieParser());
app.use(csrf({ cookie: true }));

app.get('/form', (req, res) => {
  res.send(`<form method="POST" action="/submit">
    <input type="hidden" name="_csrf" value="${req.csrfToken()}">
    <button type="submit">Submit</button>
  </form>`);
});

app.post('/submit', (req, res) => {
  res.send("Form submitted safely!");
});
```



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



Preventing XSS

Unsafe

js

```
res.send(`<h1>Hello ${req.query.name}</h1>`); // Attacker can inject <script>
```

Safe (escape output)

js

```
const escape = require('escape-html');  
app.get("/", (req, res) => {  
  res.send(`<h1>Hello ${escape(req.query.name)}</h1>`);  
});
```



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



Input Validation

```
const Joi = require('joi');

const schema = Joi.object({
  email: Joi.string().email().required(),
  age: Joi.number().min(18).max(100).required()
});

app.post("/register", (req, res) => {
  const { error } = schema.validate(req.body);
  if (error) return res.status(400).send(error.details[0].message);
  res.send("Valid input");
});
```



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



Rate Limiting

```
// limit requests
const rateLimit = require("express-rate-limit");

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP
  message: "Too many requests, try again later"
});

app.use(limiter);
```



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



Helmet Middleware

```
const helmet = require("helmet");  
app.use(helmet()); // Adds 11+ security headers  
// It prevents:  
  
// Clickjacking(X - Frame - Options)  
  
// MIME type sniffing (X-Content - Type - Options)  
  
// Cross - site scripting(Content - Security - Policy)
```



hexsecteam



hexsec_tools



hexsecteam

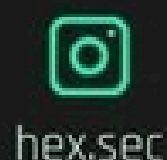
hexsec.netlify.app



Delay Login

```
//Stops brute - force attacks.  
  
let attempts = {};  
  
app.post("/login", (req, res) => {  
  const ip = req.ip;  
  if (!attempts[ip]) attempts[ip] = { count: 0, time: Date.now() };  
  
  if (attempts[ip].count > 5 && Date.now() - attempts[ip].time < 60000) {  
    return res.status(429).send("Too many attempts, try again later");  
  }  
  
  attempts[ip].count++;  
  attempts[ip].time = Date.now();  
  
  res.send("Login logic here");  
});
```

@hex.sec



Directory Traversal

```
// BAD: allows user to request any file
app.get('/files/:name', (req, res) => {
  res.sendFile(__dirname + "/uploads/" + req.params.name);
});

// GOOD: sanitize filename
const path = require("path");
app.get('/files/:name', (req, res) => {
  const safePath = path.join(__dirname, "uploads", path.basename(req.params.name));
  res.sendFile(safePath);
});
```

HTTPS

```
//Force HTTPS, even if user types http://.
```

```
app.use(helmet.hsts({
  maxAge: 31536000, // 1 year
  includeSubDomains: true,
  preload: true
}));
```



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



Content Security Policy

```
//Stops injected scripts(XSS).
```

```
app.use(helmet.contentSecurityPolicy({  
  directives: {  
    defaultSrc: ["'self'"],  
    scriptSrc: ["'self'", "trustedcdn.com"],  
    objectSrc: ["'none'"],  
    upgradeInsecureRequests: []  
  }  
}));
```



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



End-to-End Encryption

```
const crypto = require("crypto");
//Encrypt messages on sender device,
// decrypt on receiver.Server cannot read them.
const algorithm = "aes-256-cbc";
const key = crypto.randomBytes(32);
const iv = crypto.randomBytes(16);

// Encrypt
function encrypt(text) {
  const cipher = crypto.createCipheriv(algorithm, key, iv);
  return Buffer.concat([cipher.update(text), cipher.final()]).toString("hex");
}

// Decrypt
function decrypt(encrypted) {
  const decipher = crypto.createDecipheriv(algorithm, key, iv);
  return Buffer.concat([decipher.update(Buffer.from(encrypted, "hex")), decipher.final()]).toString();
}

const enc = encrypt("Secret Message");
console.log("Encrypted:", enc);
console.log("Decrypted:", decrypt(enc));
```



hexsecteam



hexsec_tools



hexsecteam

hexsec.netlify.app



Token Revocation

```
//For JWT, store refresh tokens in DB and mark
// them as invalid on logout.

// logout route
app.post("/logout", (req, res) => {
  const token = req.cookies.refreshToken;
  db.collection("tokens").deleteOne({ token }); // revoke token
  res.clearCookie("refreshToken");
  res.send("Logged out");
});
```

FOLLOW FOR MORE HACKING & AI SECURITY CONTENT
Daily tools • Tutorials • Offensive AI • Cyber Security

  Want the PDF version with all the tools in one place?
Comment “INTERESTED” and I’ll send it to you.



hexsecteam



hexsec_tools



hexsecteam